Name: Leena Hinduja
Roll No: 40

Experiment No. 2

AIM

To create and implement a Blockchain using Python with Proof-of-Work, transaction handling, SHA-256 hashing, and Flask-based APIs for mining, retrieving, and validating the blockchain.

THEORY

Blockchain
A blockchain is a distributed and immutable ledger that stores data in a sequence of connected blocks. Each block contains transactions, a timestamp, proof, and the hash of the previous block.

Genesis Block
The Genesis Block is the first block of the blockchain. Since it has no previous block, its `previous_hash` is set to `0`.

```
self.create_block(
   proof=1,
   previous_hash='0',
   transactions=[]
)
```

Proof-of-Work
Proof-of-Work (PoW) is a mining mechanism in which the program searches for a valid proof. In this experiment, the SHA-256 hash must begin with `000` before a block can be mined.

Transactions
Transactions contain the sender, receiver, and amount. They are stored in a pending transaction list and added to a block during mining.

In this experiment, a transaction is represented as:

```
{
   'sender': 'Alice',
   'receiver': 'Bob',
   'amount': 100
}
```

The create_Transactions() method adds the transaction to the pending transaction list.

```
def create_Transactions(self, sender, receiver, amount):

   transaction = {
```

```
        'sender': sender,
        'receiver': receiver,
        'amount': amount
    }

    self.transactions.append(transaction)

    return len(self.chain) + 1
```

SHA-256 Hashing

SHA-256 generates a cryptographic hash for block data. It is used to link blocks and maintain blockchain integrity.

```
def hash(self, block):

    encoded_block = json.dumps(
        block,
        sort_keys=True
    ).encode()

    return hashlib.sha256(
        encoded_block
    ).hexdigest()
```

Blockchain Validation

The blockchain is valid when each block's `previous_hash` matches the hash of the previous block and its Proof-of-Work satisfies the required `000` condition.

Flask API

Flask is used to provide APIs for adding transactions, mining blocks, displaying the blockchain, and checking its validity.

OUTPUT

Output 1 — Genesis Block

```
blockchain = Blockchain()

print(blockchain.chain)
```

```
[{'index': 1, 'timestamp': '2026-08-24 03:51:51.957873', 'proof': 1, 'previous_hash': '0'}]
```

Output 2 — Transaction Creation

```
Transaction added.
Transaction will be included in block: 2

Pending Transactions:
[{'sender': 'Alice', 'receiver': 'Bob', 'amount': 100}]
```

Output 3 — Block Mining

```
Transactions found.
Starting mining...

Block successfully mined!
{'index': 2, 'timestamp': '2026-08-24 03:59:36.381698', 'transactions': [{'sender': 'Alice', 'receiver': 'Bob', 'amount': 100}], 'proof': 533, 'previous
```

Output 4 — Pending Transction removal

```
Pending transactions:
[]
```

Output 4 — Complete Blockchain

```
Complete Blockchain:

{'index': 1, 'timestamp': '2026-08-24 03:59:08.954332', 'transactions': [], 'proof': 1, 'previous_hash': '0'}
{'index': 2, 'timestamp': '2026-08-24 03:59:36.381698', 'transactions': [{'sender': 'Alice', 'receiver': 'Bob', 'amount': 100}], 'proof': 533, 'pre
```

Output 5 — Blockchain Validation

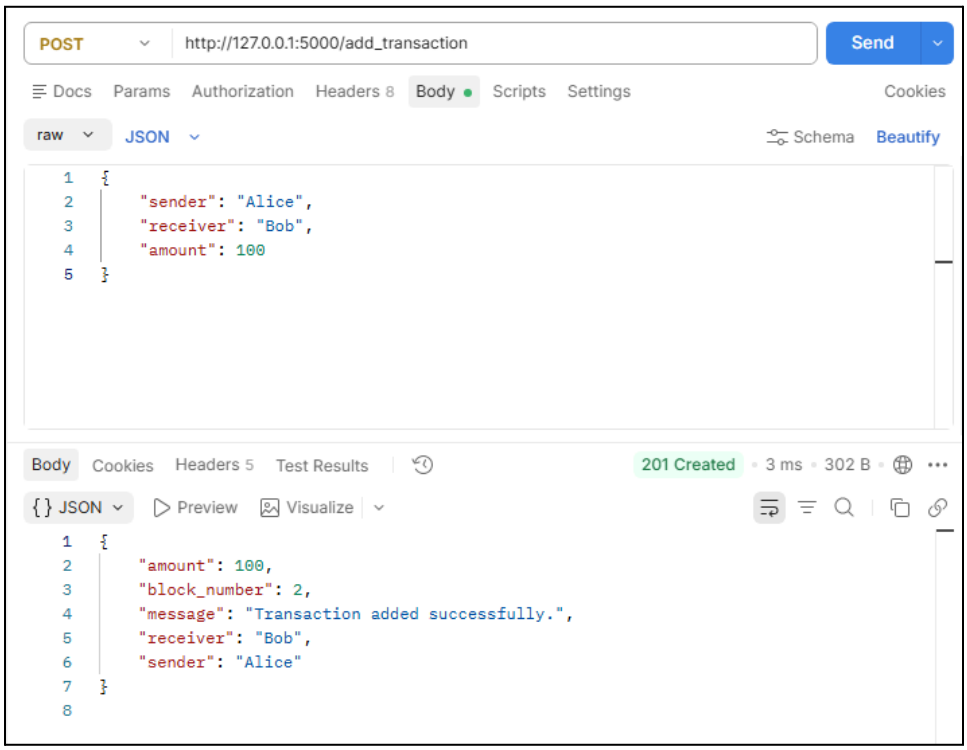
```
Blockchain valid: True
```

Output 6 — Proof Of Work

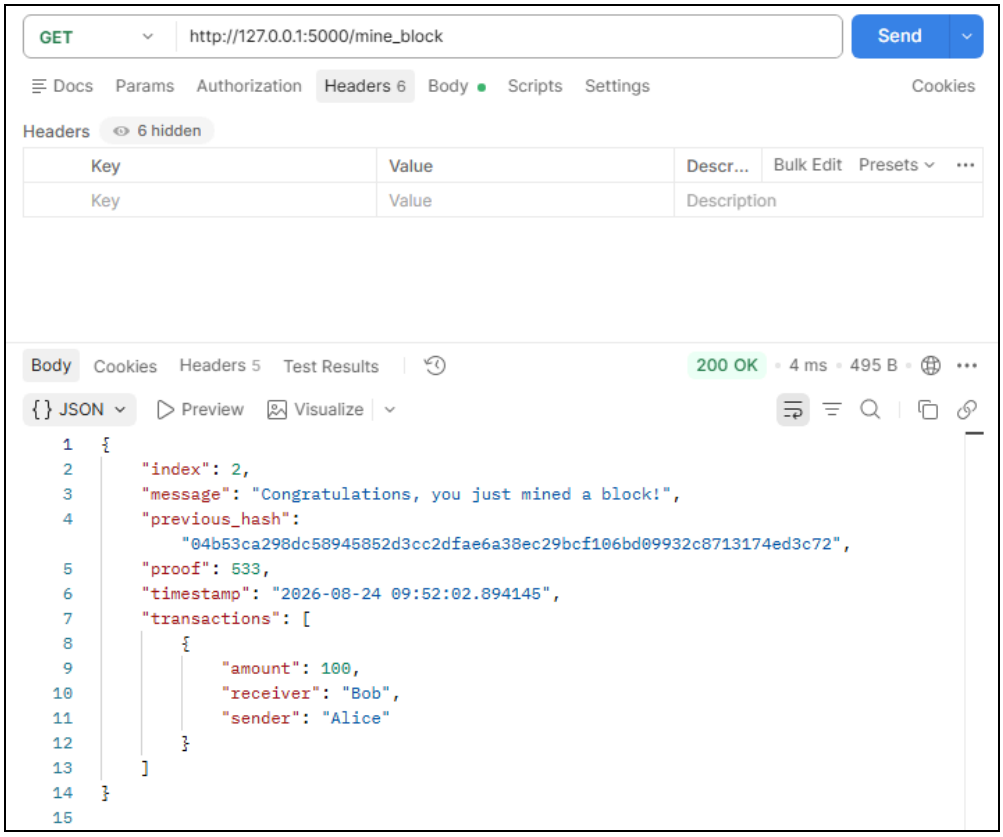
```
Proof: 533
Hash: 0000c00870f23a23ae80377298491b091db400d575be0efbde5b310f2f763ed1
First 3 characters: 000
```

```
C:\Users\Admin\Downloads\Blockchain_Experiment_2>python EXP2.py
 * Serving Flask app 'EXP2'
 * Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on http://127.0.0.1:5000
Press CTRL+C to quit
 * Restarting with stat
 * Debugger is active!
 * Debugger PIN: 690-122-618
127.0.0.1 - - [24/Aug/2026 09:50:32] "POST /add_transaction HTTP/1.1" 400 -
127.0.0.1 - - [24/Aug/2026 09:50:51] "POST /add_transaction HTTP/1.1" 201 -
127.0.0.1 - - [24/Aug/2026 09:52:02] "GET /mine_block HTTP/1.1" 200 -
127.0.0.1 - - [24/Aug/2026 09:52:37] "GET /get_chain HTTP/1.1" 200 -
127.0.0.1 - - [24/Aug/2026 09:53:29] "GET /is_valid HTTP/1.1" 200 -
```

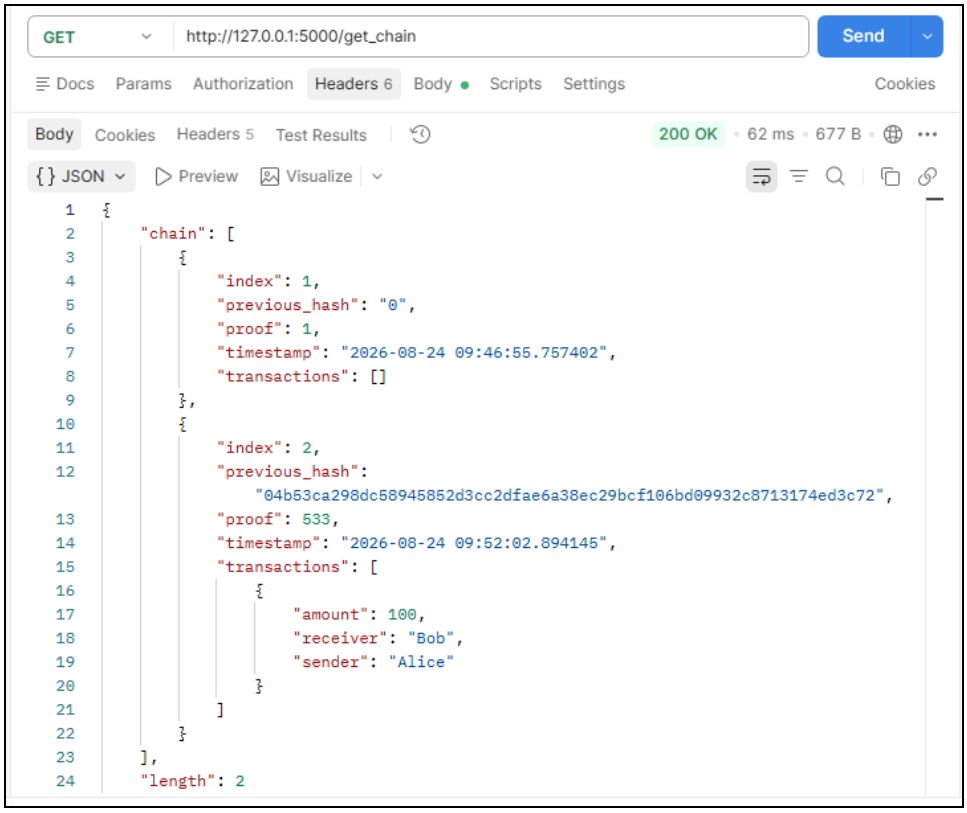
Test 1 — Add a Transaction



Test 2 —Test the Mining API



Test 3 — Get the Complete Blockchain



Test 4 —Validate the Blockchain



RESULT

The blockchain was successfully implemented using Python and Flask. Transactions were added, blocks were mined using Proof-of-Work, and the blockchain was successfully validated.

CONCLUSION

The experiment successfully demonstrated blockchain concepts including transactions, mining, SHA-256 hashing, Proof-of-Work, block linking, and blockchain validation.